

РЕФЕРАТ

Расчетно-пояснительная записка 50 с., 8 рис., 18 табл., 5 источн., 2 прил.

Ключевые слова: C#, ASP.NET Core, EF Core, bash, python, базы данных, PostgreSQL.

Цель работы — разработка базы данных для масштабируемого планирования и сохранения маршрутов в метрополитенах.

В данной работе формализована задача, выбрана структура для хранения данных, разработан механизм ролевого доступа на уровне базы данных, проведено исследование, в рамках которого выявлена зависимость времени выполнения запросов и объема хранимых данных от количества записей в таблице при наличии и отсутствии индекса.

СОДЕРЖАНИЕ

РЕФЕРАТ	2
ВВЕДЕНИЕ	3
1 Аналитический раздел	4
1.1 Анализ существующих решений	4
1.2 Формализация задачи	4
1.3 Формализация данных	7
2 Конструкторский раздел	9
2.1 Схема базы данных	9
2.2 Описание сущностей базы данных	10
2.3 Разработка триггеров	12
2.4 Ролевая модель	15
3 Технологический раздел	16
3.1 Используемое программное обеспечение	16
3.2 Реализация сущностей системы	16
3.3 Реализация ограничений сущностей	18
3.4 Реализация функций	18
3.5 Реализация ролевой модели	19
4 Исследовательская часть	21
4.1 Постановка исследования	21
4.2 Результаты исследования	22
4.2.1 Зависимость времени выполнения запроса от наличия индексов в базе данных	22
4.2.2 Использование дискового пространства	23
ЗАКЛЮЧЕНИЕ	26
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	27
ПРИЛОЖЕНИЕ А	28
ПРИЛОЖЕНИЕ В	29

ВВЕДЕНИЕ

Разрабатываемая система должна обладать конкуретным преимуществом за счет встроенной поддержки управления параметрами транспортной модели в реальном времени для поддержания актуального состояния маршрутной ситуации для наиболее правдивых поисковых запросов.

Цель – разработка базы данных для масштабируемого планирования и сохранения маршрутов в метрополитенах.

Для достижения цели поставлены следующие задачи:

- провести анализ существующих решений;
- формализовать данные;
- спроектировать схему базы данных и ограничения целостности;
- спроектировать ролевую модель на уровне базы данных;
- провести исследование зависимости времени выполнения запросов от наличия индекса в базе данных и без него;
- провести исследование зависимости объема используемого дискового пространства от наличия индекса в базе данных и без него.

1 Аналитический раздел

1.1 Анализ существующих решений

Для сравнения существующих решений были выделены следующие критерии:

- К1: Авторизация (поддерживается/не поддерживается);
- К2: История (поддерживается/не поддерживается);
- К3: Цена (платно/бесплатно);
- К4: Управление транспортной моделью в реальном времени (поддерживается/не поддерживается);

Результаты сравнения существующих решений приведены в таблице 1.1.

Таблица 1.1 – Сравнение решений

Решение	К1	К2	К3	К4
Яндекс.Метро	–	–	+	–
Google maps	+	+	+	–
Moovit	+	+	+/-	–
Transit	+	+	+	–
Metro	–	–	+/-	–
Разрабатываемое решение	+	+	+	+

Проведенный сравнительный анализ подтверждает, что разрабатываемая система обладает конкурентным преимуществом за счет поддержки динамического редактирования транспортной модели.

1.2 Формализация задачи

В рамках курсовой работы требуется спроектировать и разработать базу данных для хранения информации о схемах метро – структурах связанных сущностей таких как: ветки, станции, переезды и переходы. А также о пользователях и маршрутах, которые могут быть сохранены в истории конкретного пользователя. Необходимо разработать серверное приложение взаимодействующее с базой данных, реализующее поиск маршрутов и позволяющее изменять параметры схемы метро для пользователей с определенными правами.

На рисунке 1.1 представлена диаграмма прецедентов для разных ролей пользователей.

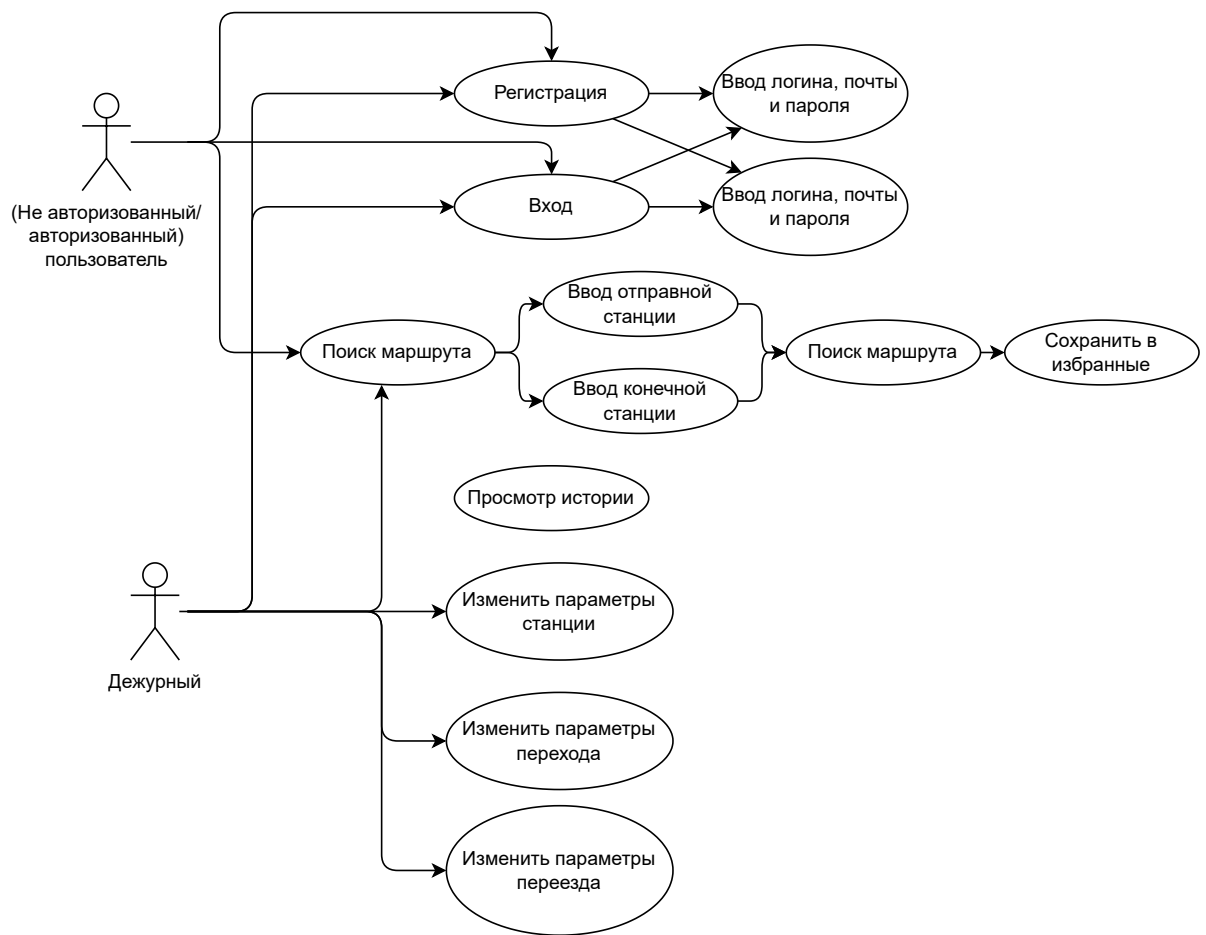


Рисунок 1.1 – Диаграмма прецедентов разных ролей

На рисунке 1.2 представлена диаграмма ключевых бизнес-процессов.

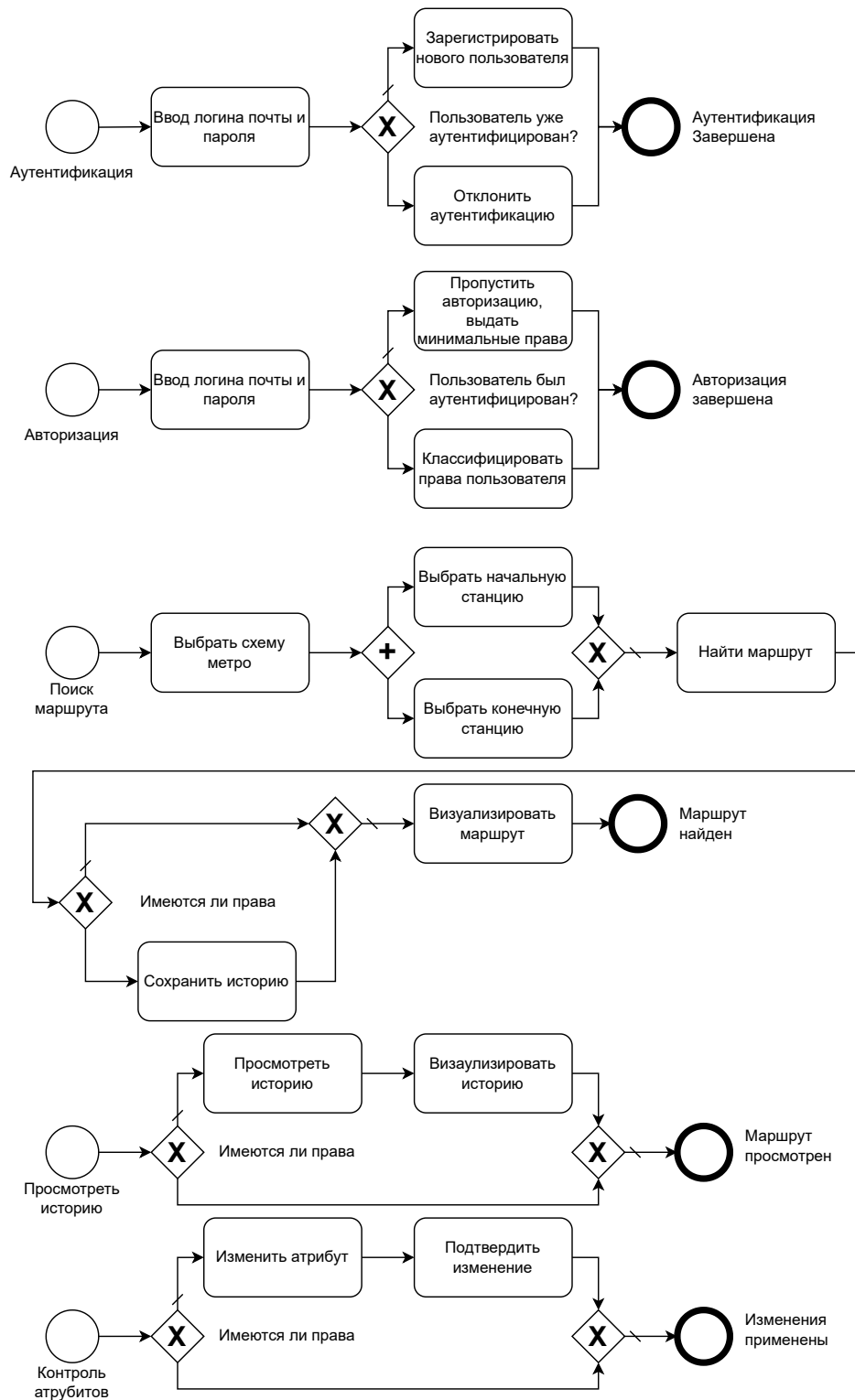


Рисунок 1.2 – Диаграмма прецедентов разных ролей

1.3 Формализация данных

База данных должна хранить информацию о:

- пользователях;
- схемах метро;
- ветках;
- станциях;
- переездах;
- переходах;
- маршрутах.

В таблице 1.2 указаны хранимые сущности и их характеристики.

Таблица 1.2 – Сущности и характеристики

сущность	характеристики
пользователь	ID, логин, пароль, почта
схема	ID, название города и схемы, SVG представление
ветка	ID, название, цвет, загруженность, доступ, время открытия/закрытия
станция	ID дежурного, название, загруженность, доступ, время открытия/закрытия
переезд	ID дежурного, загруженность, доступ, среднее время использования, время открытия/закрытия
переход	среднее время переезда
маршрут	Время создания

На рисунке 1.3 отображена диаграмма связей сущностей системы, основанная на приведенной выше таблице.

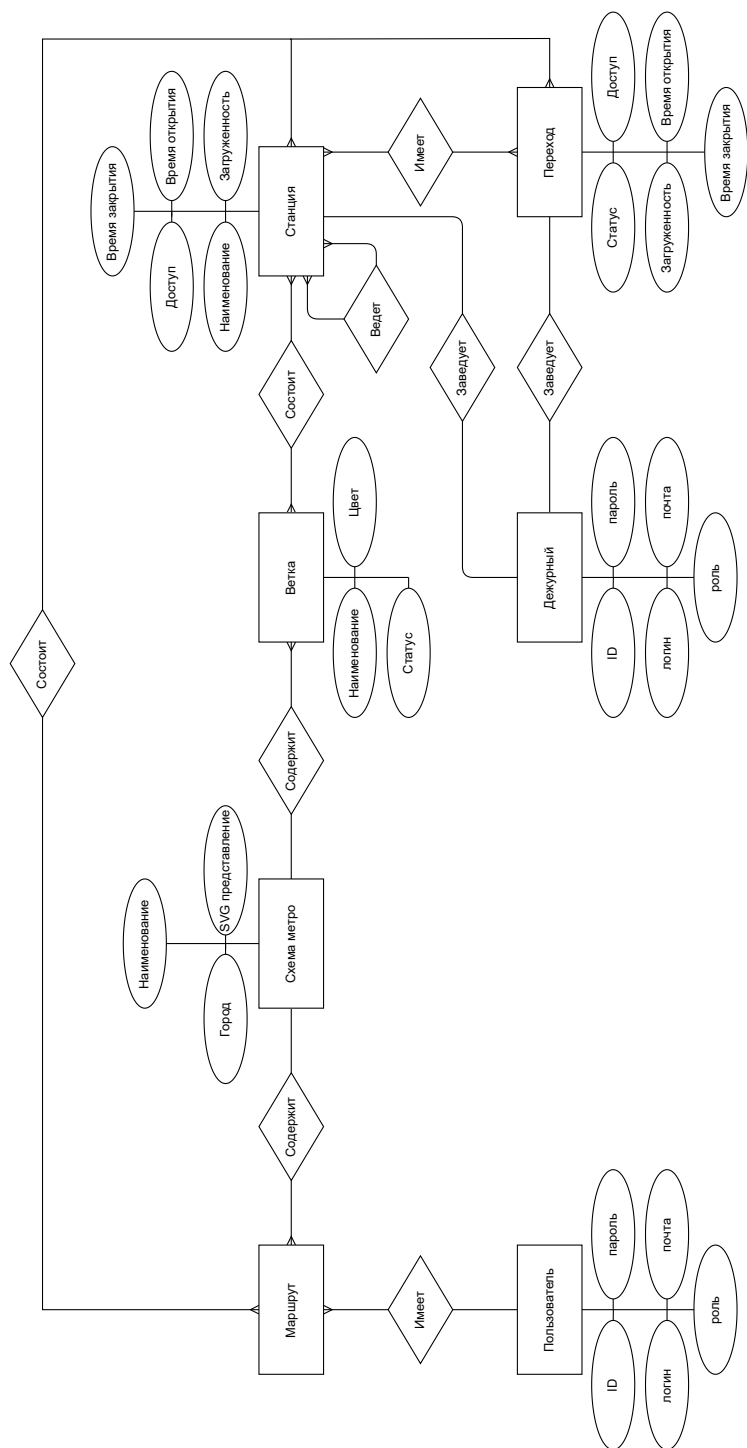


Рисунок 1.3 – ER-Диаграмма базы данных в нотации Чена

2 Конструкторский раздел

2.1 Схема базы данных

На рисунке 2.1 представлена схема базы данных.

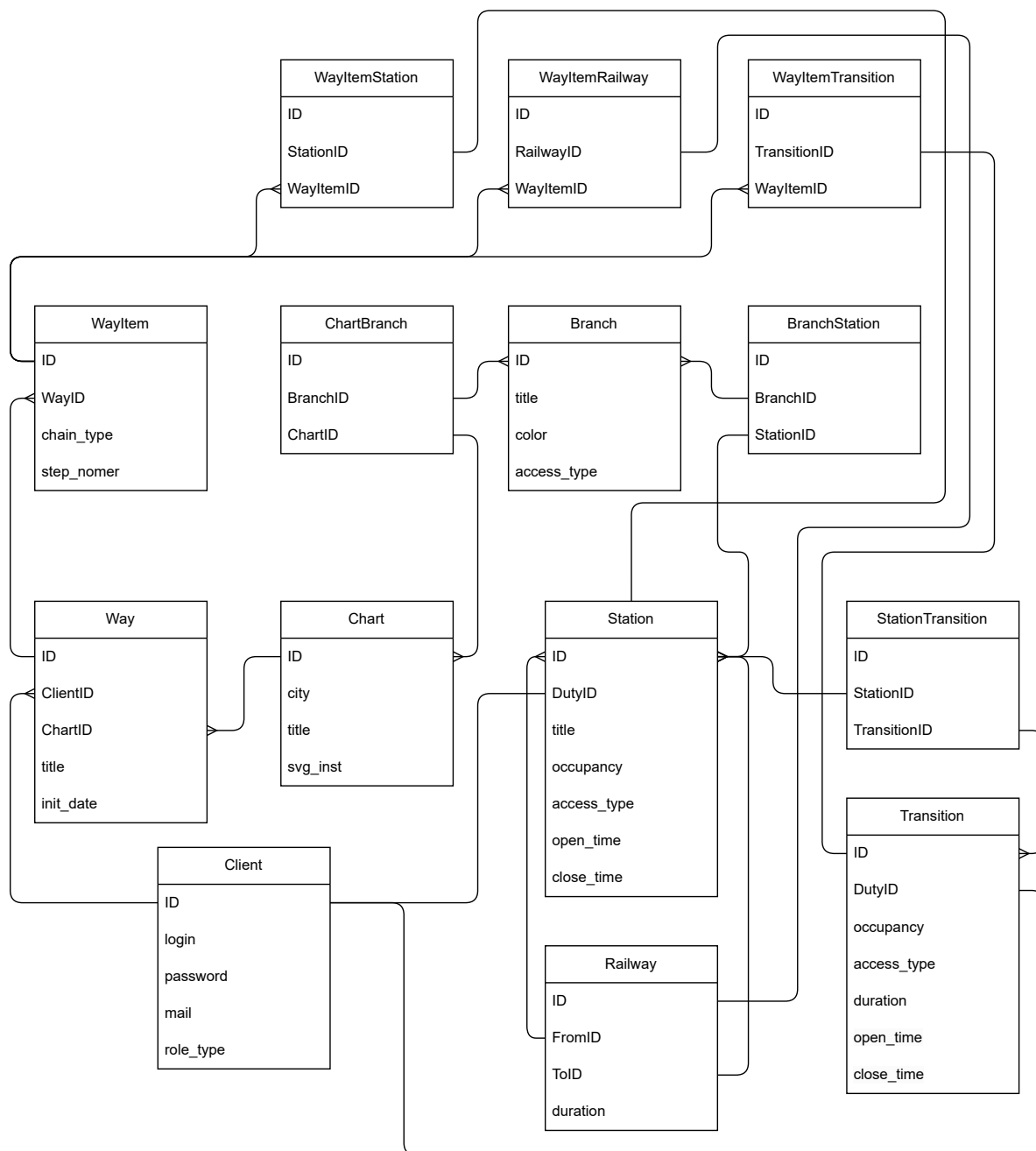


Рисунок 2.1 – Схема базы данных

2.2 Описание сущностей базы данных

Обозначения: РК — первичный ключ, FK — внешний ключ, U — уникальное поле, NN — обязательное для заполнения поле.

Описание сущностей базы данных приведено в таблицах 2.1–2.14.

Таблица 2.1 – Таблица client

поле	тип данных	ограничения	значение
id	целое число	РК	идентификатор
client_login	текст	NN, U	логин пользователя
password	текст	NN	защита
mail	дата и время	NN, U	уникальность клиента

Таблица 2.2 – Таблица chart

поле	тип данных	ограничения	значение
id	целое число	РК	идентификатор пользователя
city	текст	NN, U	название города
title	текст	NN, U	название схемы
svg_content	xml	NN	SVG представление схемы

Таблица 2.3 – Таблица branch

поле	тип данных	ограничения	значение
id	целое число	РК	идентификатор
title	текст	NN, U	название ветки
color	целое число	NN	10-е представление hex цвета
access	текст	NN	тип доступа

Таблица 2.4 – Таблица station

поле	тип данных	ограничения	значение
id	целое число	РК	идентификатор
duty_id	целое число	FK	идентификатор дежурного
title	текст	NN, U	название станции
occupancy	целое число	NN	загруженность станции
access	текст	NN	тип доступа
open_time	время без часового пояса	NN	время открытия станции
close_time	время без часового пояса	NN	время открытия станции

Таблица 2.5 – Таблица transition

поле	тип данных	ограничения	значение
id	целое число	PK	идентификатор
duty_id	целое число	FK	идентификатор дежурного
occupancy	целое число	NN	загруженность станции
access	текст	NN	тип доступа
duration	Временной интервал	NN	среднее время перехода
open_time	время без часового пояса	NN	время открытия перехода
open_time	время без часового пояса	NN	время закрытия перехода

Таблица 2.6 – Таблица railway

поле	тип данных	ограничения	значение
id	целое число	PK	идентификатор
from_id	целое число	NN, FK	связь с предыдущей станцией
to_id	целое число	NN, FK	связь со следующей станцией
duration	Временной интервал	NN	среднее время переезда

Таблица 2.7 – Таблица chart_branch

поле	тип данных	ограничения	значение
id	целое число	PK	идентификатор
chart_id	целое число	NN, FK	идентификатор родительской схемы
branch_id	целое число	NN, FK	идентификатор ветки

Таблица 2.8 – Таблица branch_station

поле	тип данных	ограничения	значение
id	целое число	PK	идентификатор
branch_id	целое число	NN, FK	идентификатор родительской ветки
station_id	целое число	NN, FK	идентификатор станции

Таблица 2.9 – Таблица station_transition

поле	тип данных	ограничения	значение
id	целое число	PK	идентификатор
station_id	целое число	NN, FK	идентификатор станции
transition_id	целое число	NN, FK	идентификатор перехода

Таблица 2.10 – Таблица way

поле	тип данных	ограничения	значение
id	целое число	PK	идентификатор
client_id	целое число	NN, FK	идентификатор пользователя
chart_id	целое число	NN, FK	идентификатор схемы
title	текст	NN, U	название маршрута
duration	Временной интервал	NN	продолжительность маршрута
init_date	Время с часовым поясом	NN	дата сохранения маршрута

Таблица 2.11 – Таблица way_item

поле	тип данных	ограничения	значение
id	целое число	PK	идентификатор
way_id	целое число	NN, FK	идентификатор родительского маршрута
nexus	текст	NN	тип связи
step_nomer	целое число	NN	номер элемента звена маршрута

Таблица 2.12 – Таблица way_item_railway

поле	тип данных	ограничения	значение
id	целое число	PK	идентификатор
way_item_id	целое число	NN, FK	идентификатор родительского элемента маршрута
railway_id	целое число	NN, FK	идентификатор переезда

Таблица 2.13 – Таблица way_item_station

поле	тип данных	ограничения	значение
id	целое число	PK	идентификатор
way_item_id	целое число	NN, FK	идентификатор родительского элемента маршрута
railway_id	целое число	NN, FK	идентификатор станции

Таблица 2.14 – Таблица way_item_transition

поле	тип данных	ограничения	значение
id	целое число	PK	идентификатор
way_item_id	целое число	NN, FK	идентификатор родительского элемента маршрута
railway_id	целое число	NN, FK	идентификатор перехода

2.3 Разработка триггеров

Для поддержания целостности базы данных необходимо спроектировать две группы триггеров:

- 1) триггеры поддерживающие целостность при обновлении или добавлении записей,
- 2) триггеры поддерживающие целостность при удалении записей.

На рисунках 2.2–2.3 представлены схемы триггеров.

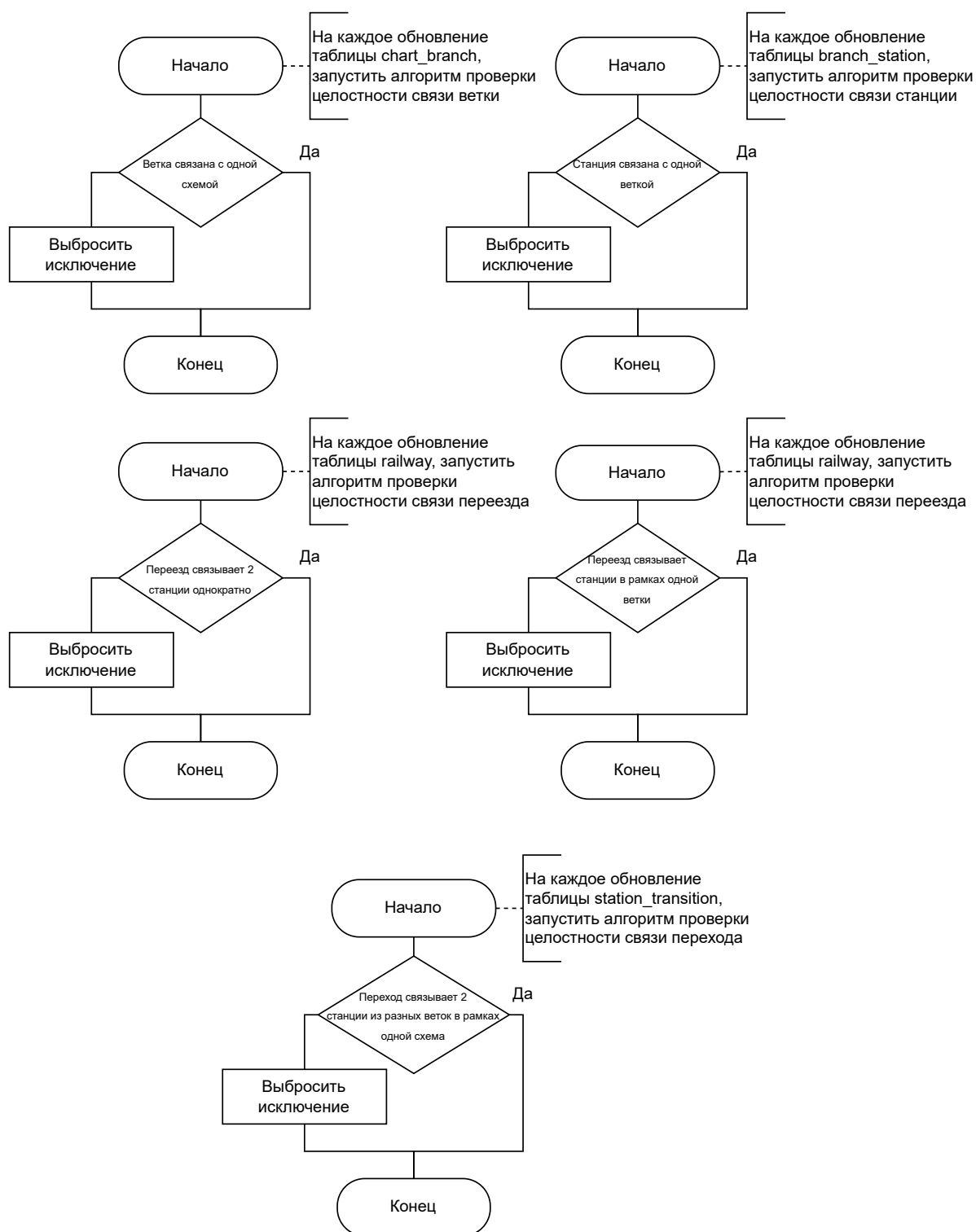


Рисунок 2.2 – Схемы алгоритмов триггеров на обновления и добавления записей

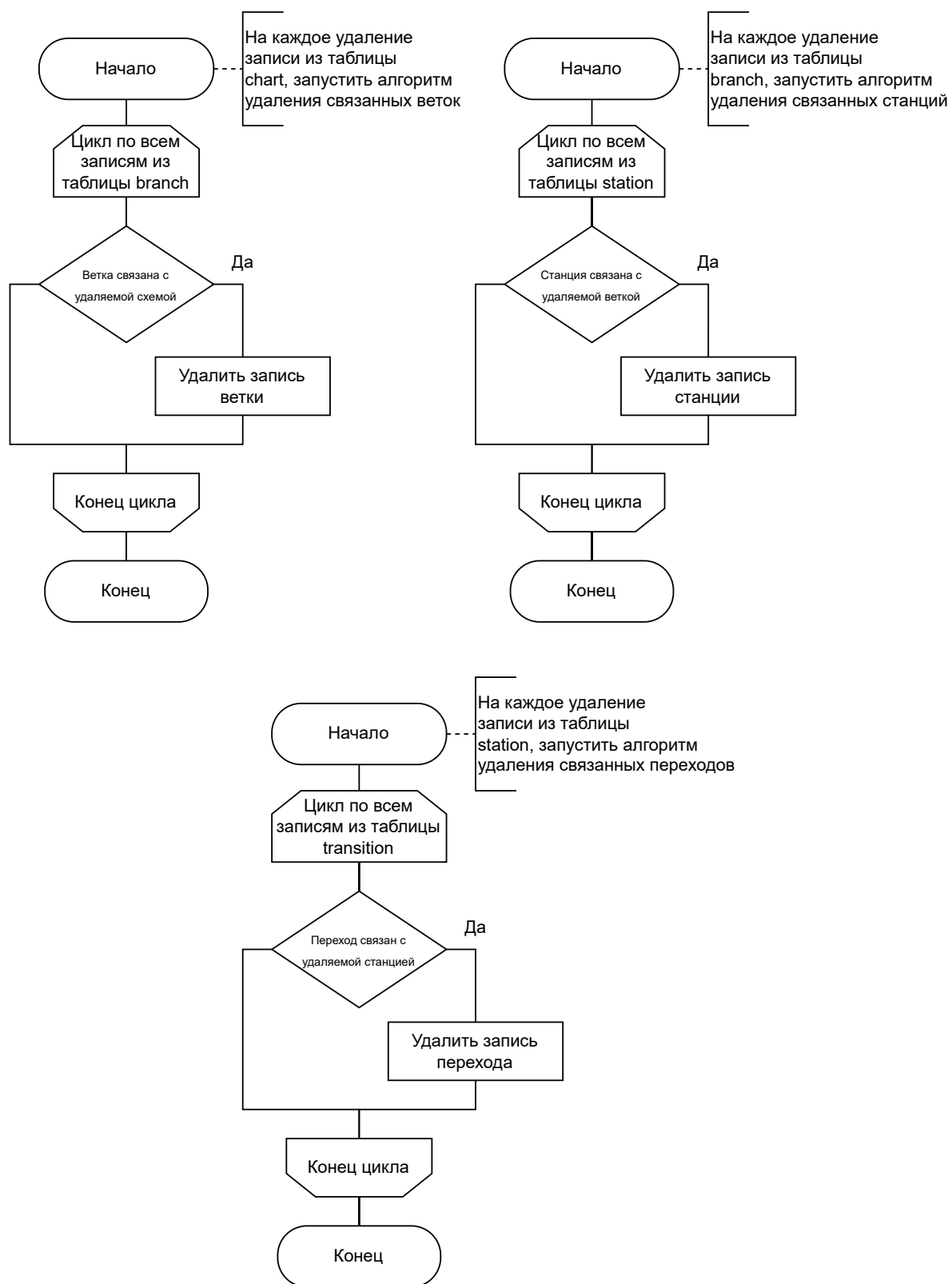


Рисунок 2.3 – Схемы алгоритмов триггеров на удаление записей

2.4 Ролевая модель

В данном разделе описаны права, которыми обладают пользователи на уровне базы данных.

Права неавторизованного пользователя:

- Чтение из таблиц client, chart, chart_branch, branch, branch_station, station, station_transition, transition, railway;
- вставка в таблицу client.

Права авторизованного пользователя:

- Чтение из таблиц client, chart, chart_branch, branch, branch_station, station, station_transition, transition, railway, way, way_item, way_item_railway, way_item_station, way_item_transition;
- Вставка в таблицы client, way, way_item, way_item_railway, way_item_station, way_item_transition.

Права дежурного:

- Чтение из таблиц client, chart, chart_branch, branch, branch_station, station, station_transition, transition, railway, way, way_item, way_item_railway, way_item_station, way_item_transition;
- Вставка в таблицы client, station, railway, transition, way, way_item, way_item_railway, way_item_station, way_item_transition.

Причем дежурный имеет доступ только к тем станциям переездам и переходам, которые связаны с ними по айди.

3 Технологический раздел

3.1 Используемое программное обеспечение

Для реализации проекта была выбрана реляционная СУБД PostgreSQL.

Серверная часть CLI-приложения реализована на C# (.NET 8) с использованием ASP.NET Core, EF Core и гексагональной архитектуры для обеспечения модульности, тестируемости и чёткого разделения ответственностей.

3.2 Реализация сущностей системы

В листингах 3.1–3.3 приведены запросы для создания таблиц.

Листинг 3.1 – Запросы для создания таблиц (часть 1)

```
1 CREATE TABLE IF NOT EXISTS client (
2     id SERIAL PRIMARY KEY,
3     client_login login_inst ,
4     client_password password_inst ,
5     mail mail_inst ,
6     privilege role_type
7 );
8
9 CREATE TABLE IF NOT EXISTS chart (
10    id SERIAL PRIMARY KEY,
11    city VARCHAR(255) ,
12    title VARCHAR(255) ,
13    svg_content XML
14 );
15
16 CREATE TABLE IF NOT EXISTS branch (
17    id SERIAL PRIMARY KEY,
18    title VARCHAR(255) ,
19    color decimal_hexcolor ,
20    access access_type
21 );
22
23 CREATE TABLE IF NOT EXISTS station (
24    id SERIAL PRIMARY KEY,
25    duty_id INT ,
26    title VARCHAR(255) ,
27    occupancy occupancy_level ,
28    access access_type ,
29    open_time TIME ,
30    close_time TIME
31 );
```


Листинг 3.2 – Запросы для создания таблиц (часть 2)

```

1 CREATE TABLE IF NOT EXISTS transition (
2     id                SERIAL PRIMARY KEY,
3     duty_id           INT,
4     occupancy          occupancy_level,
5     access             access_type,
6     duration           INTERVAL,
7     open_time          TIME,
8     close_time         TIME
9 );
10
11 CREATE TABLE IF NOT EXISTS way (
12     id                SERIAL PRIMARY KEY,
13     client_id          INT,
14     chart_id           INT,
15     title              VARCHAR(255),
16     duration           INTERVAL,
17     init_date          TIMESTAMPTZ
18 );
19
20 CREATE TABLE IF NOT EXISTS way_item (
21     id                SERIAL PRIMARY KEY,
22     way_id             INT,
23     nexus              nexus_type,
24     step_nomer         INT
25 );
26
27 CREATE TABLE IF NOT EXISTS chart_branch (
28     id                SERIAL PRIMARY KEY,
29     chart_id           INT,
30     branch_id          INT
31 );
32
33 CREATE TABLE IF NOT EXISTS branch_station (
34     id                SERIAL PRIMARY KEY,
35     branch_id          INT,
36     station_id         INT
37 );
38
39 CREATE TABLE IF NOT EXISTS railway (
40     id                SERIAL PRIMARY KEY,
41     from_id            INT,
42     to_id              INT,
43     duration           INTERVAL
44 );
45
46 CREATE TABLE IF NOT EXISTS station_transition (
47     id                SERIAL PRIMARY KEY,
48     station_id         INT,
49     transition_id      INT
50 );
51
52 CREATE TABLE IF NOT EXISTS way_item_station (
53     id                SERIAL PRIMARY KEY,
54     way_item_id        INT,
55     station_id         INT
56 );
57
58 CREATE TABLE IF NOT EXISTS way_item_railway (
59     id                SERIAL PRIMARY KEY,
60     way_item_id        INT,
61     railway_id         INT
62 );

```

Листинг 3.3 – Запросы для создания таблиц (часть 3)

```
1 CREATE TABLE IF NOT EXISTS way_item_transition (  
2     id SERIAL PRIMARY KEY,  
3     way_item_id INT,  
4     transition_id INT  
5 );
```

3.3 Реализация ограничений сущностей

В листингах 4.2–4.4 приведены запросы для создания ограничений таблиц. Для поддержки модульности настройка ограничений была вынесена в отдельный файл. Преимущественно в нем настраиваются внешние ключи и проверки уникальности определенных столбцов таблиц.

3.4 Реализация функций

В листингах 4.5–4.26 приведены реализации функций, где в листингах 4.5 – 4.6 и 4.7 – 4.10 представлены функции добавления схемы и маршрута соответственно, которые передаются как аргументы в json формате.

В листингах 4.11 – 4.19 приведено получение схемы и маршрута по айди в jsonb формате.

В листингах 4.20, 4.21 представлены вспомогательные функции конвертации цвета и поиска айди сущностей по параметрам соответственно.

В листингах 4.22 – 4.26 – триггерные функции. Они необходимы по двум причинам. Для поддержания целостности базы данных. То есть, при добавлении новой схемы, они помимо наложенных ограничений, гарантируют корректность связывания таблиц:

- Проверка уникальной связи вида схема-ветка, а именно одна и та же ветка может быть дочерней по отношению только к одной схеме;
- Проверка уникальной связи вида ветка-станция, а именно одна и та же станция может быть дочерней по отношению только к одной ветке;
- Проверка единственной связи вида станция-переезд-станция, а именно две станции могут быть связаны только одним переездом;
- Проверка локальности связи вида станция-переезд-станция, то есть станция, связанные переездом должны принадлежать одной и той же ветке;

- Проверка локальности связи вида станция-переход-станция, то есть станции, связанные переходом, должны принадлежать разным веткам, которые в свою очередь принадлежат одной и той же схеме.

Также триггеры обеспечивают корректное удаление записей из таблиц. Ввиду имени сложных связывающих таблиц, представляющие отдельные сущности, каскадное удаление не позволяет найти все взаимосвязи, поэтому требуется вручную искать связанные записи и удалять их.

3.5 Реализация ролевой модели

В листингах 3.4–3.5 приведена реализация ролевой модели.

Листинг 3.4 – Реализация ролей (часть 1)

```
1 REVOKE CREATE ON SCHEMA public FROM PUBLIC;
2
3 — UNSIGNED CLIENT —
4
5 CREATE ROLE unsigned_client WITH LOGIN PASSWORD 'qwer-tyui';
6
7 ALTER ROLE unsigned_client
8     WITH NOCREATEDB NOCREATEROLE NOREPLICATION NOINHERIT
9     VALID UNTIL 'infinity';
10
11 GRANT USAGE ON SCHEMA public TO unsigned_client;
12
13 GRANT SELECT ON
14     client ,
15     chart ,
16     chart_branch ,
17     branch ,
18     branch_station ,
19     station ,
20     station_transition ,
21     transition ,
22     railway
23 TO unsigned_client;
24
25 GRANT EXECUTE ON FUNCTION
26     get_chart_json_by_id(INT) ,
27     get_station_id_by_branch_id_station_title(INT, TEXT) ,
28     get_station_id_by_branch_title_station_title(TEXT, TEXT) ,
29     hex_color_to_int(TEXT) ,
30     int_color_to_hex(INT)
31 TO unsigned_client;
32
33 — SIGNED CLIENT —
34
35 CREATE ROLE signed_client WITH LOGIN PASSWORD 'asdf-ghjk';
36
37 ALTER ROLE signed_client
38     WITH NOCREATEDB NOCREATEROLE NOREPLICATION INHERIT
39     VALID UNTIL 'infinity';
40
41 GRANT unsigned_client TO signed_client;
```

Листинг 3.5 – Реализация ролей (часть 2)

```
43 GRANT SELECT, INSERT, UPDATE, DELETE ON
44     client ,
45     way ,
46     way_item ,
47     way_item_railway ,
48     way_item_station ,
49     way_item_transition
50 TO signed_client ;
51
52 GRANT USAGE ON SEQUENCE
53     client_id_seq ,
54     way_id_seq ,
55     way_item_id_seq ,
56     way_item_railway_id_seq ,
57     way_item_station_id_seq ,
58     way_item_transition_id_seq
59 TO signed_client ;
60
61 GRANT EXECUTE ON FUNCTION
62     add_chart_json(TEXT) ,
63     add_route_json(INT, INT, TEXT) ,
64     get_route_json_by_id(INT)
65 TO signed_client ;
66
67 — DUTY —
68
69 CREATE ROLE duty WITH LOGIN PASSWORD 'zxcv-bnml';
70
71 ALTER ROLE duty
72     WITH NOCREATEDB NOCREATEROLE NOREPLICATION INHERIT
73     VALID UNTIL 'infinity';
74
75 GRANT signed_client TO duty;
76
77 GRANT UPDATE ON
78     station ,
79     transition ,
80     railway
81 TO duty;
```

4 Исследовательская часть

В данном разделе исследуется влияние наличия индекса на время выполнения запроса выборки данных из таблиц и объем дискового пространства, занимаемого хранимыми данными.

4.1 Постановка исследования

За основу выбирается функция добавления схемы (листинги 4.5 – 4.6).

В листинге 4.1 приведен запрос для создания индексов.

Листинг 4.1 – запрос для создания индексов

```
1 CREATE INDEX IF NOT EXISTS idx_branch_station_branch_id ON
   branch_station(branch_id);
2 CREATE INDEX IF NOT EXISTS idx_station_title ON station(title);
3
4 CREATE INDEX IF NOT EXISTS idx_branch_station_branch_id_station_id ON
   branch_station(branch_id, station_id);
5
6 CREATE INDEX IF NOT EXISTS idx_branch_title ON branch(title);
7
8 CREATE INDEX IF NOT EXISTS idx_chart_branch_chart_id ON chart_branch(chart_id);
9 CREATE INDEX IF NOT EXISTS idx_chart_branch_branch_id ON chart_branch(branch_id);
10
11 CREATE INDEX IF NOT EXISTS idx_branch_station_station_id ON
   branch_station(station_id);
12
13 CREATE INDEX IF NOT EXISTS idx_station_transition_station_id ON
   station_transition(station_id);
14 CREATE INDEX IF NOT EXISTS idx_station_transition_transition_id ON
   station_transition(transition_id);
15
16 CREATE INDEX IF NOT EXISTS idx_railway_from_id ON railway(from_id);
17 CREATE INDEX IF NOT EXISTS idx_railway_to_id ON railway(to_id);
18
19 CREATE INDEX IF NOT EXISTS idx_chart_city ON chart(city);
20 CREATE INDEX IF NOT EXISTS idx_chart_title ON chart(title);
21
22 CREATE INDEX IF NOT EXISTS idx_station_occupancy ON station(occupancy);
23 CREATE INDEX IF NOT EXISTS idx_station_open_time ON station(open_time);
24 CREATE INDEX IF NOT EXISTS idx_station_close_time ON station(close_time);
25
26 CREATE INDEX IF NOT EXISTS idx_transition_duration ON transition(duration);
27 CREATE INDEX IF NOT EXISTS idx_transition_access ON transition(access);
28 CREATE INDEX IF NOT EXISTS idx_transition_open_time ON transition(open_time);
29 CREATE INDEX IF NOT EXISTS idx_transition_close_time ON transition(close_time);
```

Технические характеристики устройства, на котором выполнялись замеры времени:

- операционная система — Ubuntu 22.04.1 LTS Release: 22.04;
- оперативная память — 16 ГБ, 3200 МГц;
- процессор — AMD Ryzen 7 4800H with Radeon Graphics, 2900 МГц, ядер: 16.

4.2 Результаты исследования

4.2.1 Зависимость времени выполнения запроса от наличия индексов в базе данных

На рисунке 4.1 представлен график зависимости времени выполнения запроса от количества записей и от наличия индекса.

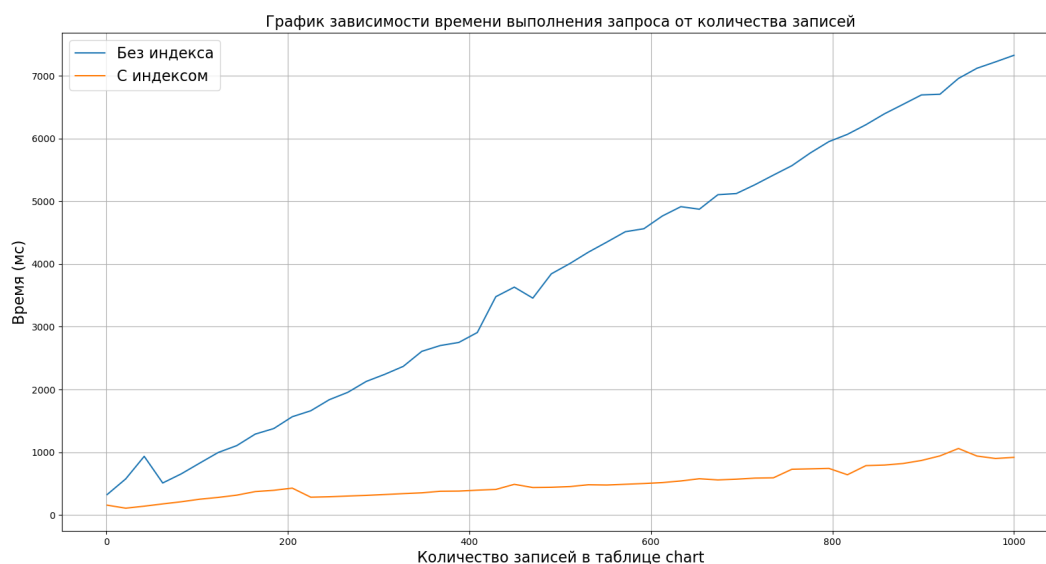


Рисунок 4.1 – График зависимости выполнения запроса от количества записей

Таблица 4.1 – Результаты замеров времени выполнения запросов

Кол-во записей	Без индекса (мс)	С индексом (мс)
50	684	154
100	796	241
150	1 118	332
200	1 499	419
250	1 837	301
300	2 196	323
350	2 626	355
400	2 847	382
450	3 630	463
500	3 932	456
550	4 356	477
600	4 597	505
650	4 868	657
700	5 167	572
750	5 548	722
800	6 005	634
850	6 370	790
900	6 628	842
950	6 953	946
1000	7 326	918

Результаты показывают, что при увеличении количества записей в таблице с индексом время выполнения запроса практически не меняется, в то время как при отсутствии индекса наблюдается увеличение времени выполнения запроса.

4.2.2 Использование дискового пространства

Для определения занимаемого таблицами пространства использовалась функция PostgreSQL – *pg_total_relation_size*.

На рисунке 4.2 представлен график зависимости объема дискового пространства от количества записей в таблице *chart* при наличии и отсутствии индекса в базе данных.

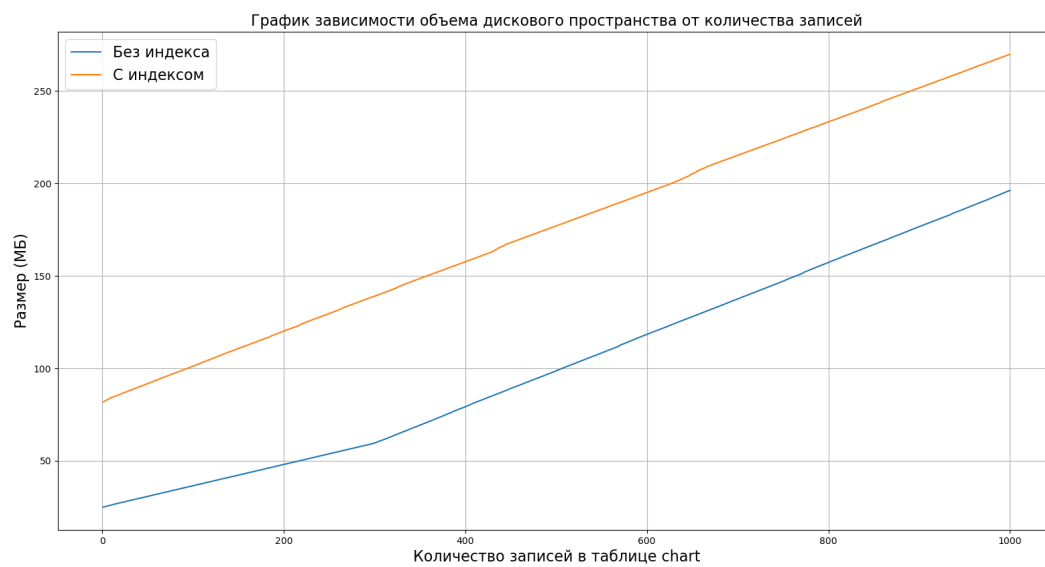


Рисунок 4.2 – График зависимости времени выполнения запроса от количества записей

Таблица 4.2 – Результаты замеров объема занимаемой памяти

Кол-во записей	Без индекса (МБ)	С индексом (МБ)
50	30.82	38.52
100	36.58	45.72
150	42.31	52.88
200	48.05	60.06
250	53.81	67.26
300	59.66	74.57
350	69.32	86.64
400	79.29	99.11
450	89.11	111.38
500	98.66	123.32
550	108.32	135.39
600	118.34	147.92
650	127.92	159.9
700	137.53	171.91
750	147.53	183.85
800	157.25	196.56
850	166.90	208.62
900	176.57	220.71
950	186.23	232.78
1000	196.16	245.2

Вывод

Использование индексов позволяет оптимизировать время выполнения запроса к базе данных. Индексы занимают дополнительное дисковое пространство, но при это позволяют уменьшить время выполнения запроса. При максимальном количестве записей (1000) время выполнения запроса с использованием индексов уменьшилось более чем в 7 раз, в то время как занимаемое дисковое пространство увеличилось примерно на 25%.

ЗАКЛЮЧЕНИЕ

Цель достигнута — разработана база данных для хранения и обработки данных маркетплейса.

Были выполнены все задачи:

- проведен анализ существующих решений;
- формализованы данные;
- спроектирована схема базы данных и ограничения целостности;
- спроектирована ролевая модель на уровне базы данных;
- проведено сравнение времени выполнения запросов при наличии индекса в базе данных и без него;
- проведено сравнение объема используемого дискового пространства при наличии индекса в базе данных и без него.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Web-разработки в asp. Net web forms : учебник для вузов / С. Т. Гуляева, В. В. Миронов, Н. О. Котелина, И. И. Лавреш. — Москва : Издательство Юрайт, 2025. — 134 с. — (Высшее образование). — ISBN 978-5-534-19885-0.
2. Стружкин, Н. П. Базы данных: проектирование : учебник для вузов / Н. П. Стружкин, В. В. Годин. — Москва : Издательство Юрайт, 2025. — 477 с. — (Высшее образование). — ISBN 978-5-534-00229-4.
bibitemBD2 Илюшечкин, В. М. Основы использования и проектирования баз данных : учебник для вузов / В. М. Илюшечкин. — Москва : Издательство Юрайт, 2025. — 213 с. — (Высшее образование). — ISBN 978-5-534-03617-6.
3. Пирогов, В. Ю. Основы программирования на языке Bash : учебное пособие / В. Ю. Пирогов. — Москва : ФЛИНТА, 2024. — 94 с. — ISBN 978-5-9765-5653-9.
4. PostgreSQL - The World's Most Advanced Open Source Relational Database [Электронный ресурс]. — URL: <https://www.postgresql.org/> (Дата обращения 07.09.2025)

ПРИЛОЖЕНИЕ А

Презентация к курсовой работе

Презентация содержит 11 слайдов.

ПРИЛОЖЕНИЕ В

Листинг к курсовой работе

Листинг 4.2 – Запросы для создания ограничений таблиц (часть 1)

```
1 ALTER TABLE client
2     ALTER COLUMN client_login SET NOT NULL,
3     ALTER COLUMN client_password SET NOT NULL,
4     ALTER COLUMN mail SET NOT NULL,
5     ALTER COLUMN privilege SET DEFAULT 'UNSIGNED',
6     ADD CONSTRAINT uk_client_login
7         UNIQUE (client_login),
8     ADD CONSTRAINT uk_client_mail
9         UNIQUE (mail);
10
11 ALTER TABLE chart
12     ALTER COLUMN city SET NOT NULL,
13     ALTER COLUMN title SET NOT NULL,
14     ALTER COLUMN svg_content SET DEFAULT '<svg width="200" height="200"><text
15         y="16">Пустая схема</text></svg>',
16     ADD CONSTRAINT ck_valid_svg
17         CHECK (svg_content::xml IS DOCUMENT),
18     ADD CONSTRAINT uk_chart_city_title
19         UNIQUE (city, title);
20
21 ALTER TABLE branch
22     ALTER COLUMN title SET NOT NULL,
23     ALTER COLUMN color SET DEFAULT 0,
24     ALTER COLUMN access SET DEFAULT 'ACCESSIBLE';
25
26 ALTER TABLE station
27     ALTER COLUMN duty_id DROP NOT NULL,
28     ALTER COLUMN title SET NOT NULL,
29     ALTER COLUMN occupancy SET DEFAULT 5,
30     ALTER COLUMN access SET DEFAULT 'ACCESSIBLE',
31     ALTER COLUMN open_time SET NOT NULL,
32     ALTER COLUMN close_time SET NOT NULL,
33     ADD CONSTRAINT ck_station_different_open_close_time
34         CHECK (open_time <> close_time),
35     ADD CONSTRAINT fk_station_duty_id
36         FOREIGN KEY (duty_id) REFERENCES client (id) ON DELETE SET NULL;
37
38 ALTER TABLE transition
39     ALTER COLUMN duty_id DROP NOT NULL,
40     ALTER COLUMN occupancy SET DEFAULT 5,
41     ALTER COLUMN access SET DEFAULT 'ACCESSIBLE',
42     ALTER COLUMN duration SET NOT NULL,
43     ALTER COLUMN open_time SET NOT NULL,
44     ALTER COLUMN close_time SET NOT NULL,
45     ADD CONSTRAINT ck_transition_different_open_close_time
46         CHECK (open_time <> close_time),
47     ADD CONSTRAINT fk_transition_duty_id
48         FOREIGN KEY (duty_id) REFERENCES client (id) ON DELETE SET NULL;
```

Листинг 4.3 – Запросы для создания ограничений таблиц (часть 2)

```

1 ALTER TABLE way
2     ALTER COLUMN client_id SET NOT NULL,
3     ALTER COLUMN chart_id SET NOT NULL,
4     ALTER COLUMN title SET NOT NULL,
5     ALTER COLUMN init_date SET NOT NULL,
6     ADD CONSTRAINT uk_way_client_id_title
7         UNIQUE (client_id, title),
8     ADD CONSTRAINT fk_way_client_id
9         FOREIGN KEY (client_id) REFERENCES client (id) ON DELETE CASCADE,
10    ADD CONSTRAINT fk_way_chart_id
11        FOREIGN KEY (chart_id) REFERENCES chart (id) ON DELETE CASCADE;
12
13
14 ALTER TABLE way_item
15     ALTER COLUMN way_id SET NOT NULL,
16     ALTER COLUMN nexus SET NOT NULL,
17     ALTER COLUMN step_nomer SET NOT NULL,
18     ADD CONSTRAINT uk_way_item_way_id_step_nomer
19         UNIQUE (way_id, step_nomer),
20     ADD CONSTRAINT fk_way_item_way_id
21         FOREIGN KEY (way_id) REFERENCES way (id) ON DELETE CASCADE;
22
23
24 ALTER TABLE chart_branch
25     ALTER COLUMN chart_id SET NOT NULL,
26     ALTER COLUMN branch_id SET NOT NULL,
27     ADD CONSTRAINT uk_chart_branch_chart_id_branch_id
28         UNIQUE (chart_id, branch_id), — Ни одна схема не имеет дубликатов веток
29     ADD CONSTRAINT uk_chart_branch_branch_id
30         UNIQUE (branch_id), — Каждая ветка связывается со схемой только один раз
31     ADD CONSTRAINT fk_chart_branch_chart_id
32         FOREIGN KEY (chart_id) REFERENCES chart (id) ON DELETE CASCADE,
33     ADD CONSTRAINT fk_chart_branch_branch_id
34         FOREIGN KEY (branch_id) REFERENCES branch (id) ON DELETE CASCADE;
35
36
37 ALTER TABLE branch_station
38     ALTER COLUMN branch_id SET NOT NULL,
39     ALTER COLUMN station_id SET NOT NULL,
40     ADD CONSTRAINT uk_branch_station_branch_id_station_id
41         UNIQUE (branch_id, station_id), — Ни одна ветка не имеет дубликатов станций
42     ADD CONSTRAINT uk_branch_station_station_id
43         UNIQUE (station_id), — Каждая станция связывается с веткой только один раз
44     ADD CONSTRAINT fk_branch_station_branch_id
45         FOREIGN KEY (branch_id) REFERENCES branch (id) ON DELETE CASCADE,
46     ADD CONSTRAINT fk_branch_station_station_id
47         FOREIGN KEY (station_id) REFERENCES station (id) ON DELETE CASCADE;
48
49
50 ALTER TABLE railway
51     ALTER COLUMN from_id SET NOT NULL,
52     ALTER COLUMN to_id SET NOT NULL,
53     ALTER COLUMN duration SET NOT NULL,
54     ADD CONSTRAINT uk_railway_from_id_to_id
55         UNIQUE (from_id, to_id),
56     ADD CONSTRAINT ck_railway_different_stations
57         CHECK (from_id <> to_id),
58     ADD CONSTRAINT fk_railway_from_id
59         FOREIGN KEY (from_id) REFERENCES station (id) ON DELETE CASCADE,
60     ADD CONSTRAINT fk_railway_to_id
61         FOREIGN KEY (to_id) REFERENCES station (id) ON DELETE CASCADE;

```

Листинг 4.4 – Запросы для создания ограничений таблиц (часть 3)

```

1 ALTER TABLE station_transition
2     ALTER COLUMN station_id SET NOT NULL,
3     ALTER COLUMN transition_id SET NOT NULL,
4     ADD CONSTRAINT uk_station_transition_station_id_transition_id
5         UNIQUE (station_id, transition_id),
6     ADD CONSTRAINT fk_station_transition_station_id
7         FOREIGN KEY (station_id) REFERENCES station (id) ON DELETE CASCADE,
8     ADD CONSTRAINT fk_station_transition_transition_id
9         FOREIGN KEY (transition_id) REFERENCES transition (id) ON DELETE CASCADE;
10
11
12 ALTER TABLE way_item_station
13     ALTER COLUMN way_item_id SET NOT NULL,
14     ALTER COLUMN station_id SET NOT NULL,
15     ADD CONSTRAINT uk_way_item_station_way_item_id_station_id
16         UNIQUE (way_item_id, station_id),
17     ADD CONSTRAINT uk_way_item_station_way_item_id
18         UNIQUE (way_item_id),
19     ADD CONSTRAINT fk_way_item_station_way_item_id
20         FOREIGN KEY (way_item_id) REFERENCES way_item (id) ON DELETE CASCADE,
21     ADD CONSTRAINT fk_way_item_station_station_id
22         FOREIGN KEY (station_id) REFERENCES station (id) ON DELETE CASCADE;
23
24
25 ALTER TABLE way_item_railway
26     ALTER COLUMN way_item_id SET NOT NULL,
27     ALTER COLUMN railway_id SET NOT NULL,
28     ADD CONSTRAINT uk_way_item_railway_way_item_id_railway_id
29         UNIQUE (way_item_id, railway_id),
30     ADD CONSTRAINT uk_way_item_railway_way_item_id
31         UNIQUE (way_item_id),
32     ADD CONSTRAINT fk_way_item_railway_way_item_id
33         FOREIGN KEY (way_item_id) REFERENCES way_item (id) ON DELETE CASCADE,
34     ADD CONSTRAINT fk_way_item_railway_railway_id
35         FOREIGN KEY (railway_id) REFERENCES railway (id) ON DELETE CASCADE;
36
37
38 ALTER TABLE way_item_transition
39     ALTER COLUMN way_item_id SET NOT NULL,
40     ALTER COLUMN transition_id SET NOT NULL,
41     ADD CONSTRAINT uk_way_item_transition_way_item_id_transition_id
42         UNIQUE (way_item_id, transition_id),
43     ADD CONSTRAINT uk_way_item_transition_way_item_id
44         UNIQUE (way_item_id),
45     ADD CONSTRAINT fk_way_item_transition_way_item_id
46         FOREIGN KEY (way_item_id) REFERENCES way_item (id) ON DELETE CASCADE,
47     ADD CONSTRAINT fk_way_item_transition_transition_id
48         FOREIGN KEY (transition_id) REFERENCES transition (id) ON DELETE CASCADE;

```

Листинг 4.5 – Реализация функции добавления схемы (часть 1)

```

1 CREATE OR REPLACE FUNCTION add_chart_json(
2   p_json_data TEXT
3 ) RETURNS INT AS $$
4 DECLARE
5   v_json_data JSONB := p_json_data::JSONB;
6
7   v_chart_id      INT;  — айди созданной записи схемы
8   v_branch_id     INT;  — айди созданной записи ветки
9   v_station_id    INT;  — айди созданной записи станции
10  v_transition_id  INT;  — айди созданной записи перехода
11
12  v_branch_item    RECORD;
13  v_station_item   RECORD;
14  v_railway_item   RECORD;
15  v_transition_item RECORD;
16 BEGIN
17   IF p_json_data IS NULL OR NOT (p_json_data ~ '^[\\s]*\\{.*\\}[\\s]*$') THEN
18     RAISE EXCEPTION 'Некорректный JSON: входные данные пусты или не объект';
19   END IF;
20
21   INSERT INTO chart(city, title)
22   VALUES(
23     v_json_data->>'city',
24     v_json_data->>'title'
25   )
26   RETURNING id INTO v_chart_id;
27
28   FOR v_branch_item IN
29     SELECT * FROM jsonb_array_elements(v_json_data->'branches')
30   LOOP
31
32     INSERT INTO branch(title, color, access)
33     VALUES(
34       (v_branch_item.value)->>'title',
35       hex_color_to_int((v_branch_item.value)->>'color'),
36       ((v_branch_item.value)->>'accesstype')::access_type
37     )
38     RETURNING id INTO v_branch_id;
39
40     INSERT INTO chart_branch(chart_id, branch_id)
41     VALUES(
42       v_chart_id,
43       v_branch_id
44     );
45
46     FOR v_station_item IN
47       SELECT * FROM jsonb_array_elements((v_branch_item.value)->'stations')
48     LOOP
49
50       INSERT INTO station(title, occupancy, access, open_time, close_time)
51       VALUES(
52         (v_station_item.value)->>'title',
53         ((v_station_item.value)->>'occupancy')::SMALLINT,
54         ((v_station_item.value)->>'accesstype')::access_type,
55         ((v_station_item.value)->>'opentime')::TIME,
56         ((v_station_item.value)->>'closetime')::TIME
57       )
58       RETURNING id INTO v_station_id;

```


Листинг 4.6 – Реализация функции добавления схемы (часть 2)

```

59      INSERT INTO branch_station(branch_id, station_id)
60      VALUES(
61          v_branch_id,
62          v_station_id
63      );
64
65  END LOOP;
66
67  FOR v_railway_item IN
68      SELECT * FROM jsonb_array_elements((v_branch_item.value)->'railways')
69  LOOP
70
71      INSERT INTO railway(from_id, to_id, duration)
72      VALUES(
73          get_station_id_by_branch_id_station_title(v_branch_id,
74              (v_railway_item.value)->>'from'),
75          get_station_id_by_branch_id_station_title(v_branch_id,
76              (v_railway_item.value)->>'to'),
77          ((v_railway_item.value)->>'duration')::INTERVAL
78      );
79
80  END LOOP;
81
82  END LOOP;
83
84  FOR v_transition_item IN
85      SELECT * FROM jsonb_array_elements(v_json_data->'transitions')
86  LOOP
87
88      INSERT INTO transition(occupancy, access, duration, open_time, close_time)
89      VALUES(
90          ((v_transition_item.value)->>'occupancy')::SMALLINT,
91          ((v_transition_item.value)->>'accesstype')::access_type,
92          ((v_transition_item.value)->>'duration')::INTERVAL,
93          ((v_transition_item.value)->>'opentime')::TIME,
94          ((v_transition_item.value)->>'closetime')::TIME
95      )
96      RETURNING id INTO v_transition_id;
97
98      INSERT INTO station_transition(station_id, transition_id)
99      VALUES (
100          get_station_id_by_branch_title_station_title((v_transition_item.value)
101              ->>'branchsrc', (v_transition_item.value)->>'stationsrc'),
102          v_transition_id
103      );
104
105      INSERT INTO station_transition(station_id, transition_id)
106      VALUES (
107          get_station_id_by_branch_title_station_title((v_transition_item.value)
108              ->>'branchdst', (v_transition_item.value)->>'stationdst'),
109          v_transition_id
110      );
111
112  END LOOP;
113
114  RETURN v_chart_id;
115 END;
116 $$ LANGUAGE plpgsql;

```

Листинг 4.7 – Реализация функции добавления маршрута (часть 1)

```

1 CREATE OR REPLACE FUNCTION add_route_json(
2     p_client_id INT,
3     p_chart_id INT,
4     p_json_data TEXT
5 ) RETURNS INT AS $$
6 DECLARE
7     v_json_data JSONB := p_json_data::JSONB;
8
9     v_way_id INT;           — айди созданной записи маршрута
10    v_item RECORD;         — обход массива звеньев маршрута
11    v_way_item_id INT;      — айди созданного звена
12    v_step_nomer INT := 1;  — номер звена маршрута
13    v_branch_id INT;        — айди найденной ветки
14    v_station_id INT;       — айди найденной станции
15    v_station_from_id INT;  — айди станции: <ОТ>
16    v_station_to_id INT;    — айди станции: <НА>
17    v_railway_id INT;       — айди найденного переезда
18    v_transition_id INT;    — айди найденного перехода
19 BEGIN
20     — Проверка конвертации
21     IF p_json_data IS NULL OR NOT (p_json_data ~ '^[\\s]*\\{.*\\}[\\s]*$') THEN
22         RAISE EXCEPTION 'Некорректный JSON: входные данные пусты или не объект';
23     END IF;
24
25     — Создать запись нового маршрута
26     INSERT INTO way (client_id, chart_id, title, duration, init_date)
27     VALUES (
28         p_client_id,
29         p_chart_id,
30         v_json_data->>'Title',
31         (v_json_data->>'Duration')::INTERVAL,
32         date_trunc('second', NOW())::TIMESTAMPTZ
33     )
34     RETURNING id INTO v_way_id;
35
36     — Вставка звеньев маршрута
37     FOR v_item IN SELECT * FROM jsonb_array_elements(v_json_data->'RouteItems')
38     LOOP
39         — Создать запись о звенье маршрута
40         INSERT INTO way_item (way_id, nexus, step_nomer)
41         VALUES (
42             v_way_id,
43             CASE
44                 WHEN (v_item.value)->>'$type' = 'station' THEN 'STATION'::nexus_type
45                 WHEN (v_item.value)->>'$type' = 'railway' THEN 'RAILWAY'::nexus_type
46                 WHEN (v_item.value)->>'$type' = 'transition' THEN
47                     'TRANSITION'::nexus_type
48             END,
49             v_step_nomer
50         )
51         RETURNING id INTO v_way_item_id;
52
53         — Конкретизация и связывание звяна с реальными станциями, переездами и переход
54         а схемы
55         CASE
56             — Станция
57             WHEN (v_item.value)->>'$type' = 'station' THEN
58                 — Поиск связываемой ветки (обновлять айди нужно только на каждую станц
59                 ию и переход)
60                 SELECT
61                     b.id INTO v_branch_id
62                 FROM
63                     branch AS b
64                 WHERE

```

Листинг 4.8 – Реализация функции добавления маршрута (часть 2)

```

62         b.title = (v_item.value)-->>'BranchTitle' AND
63         EXISTS(
64             SELECT
65                 1
66             FROM
67                 chart_branch AS cb
68             WHERE
69                 cb.chart_id = p_chart_id AND cb.branch_id = b.id
70         );
71
72     — Поиск связываемой станции
73     SELECT
74         s.id INTO v_station_id
75     FROM
76         station AS s
77     INNER JOIN
78         branch_station AS bs ON s.id = bs.station_id
79     WHERE
80         bs.branch_id = v_branch_id AND
81         s.title = (v_item.value)-->>'Title';
82
83     — Вставка звена маршрута <Станция>
84     INSERT INTO way_item_station (way_item_id, station_id)
85     VALUES (v_way_item_id, v_station_id);
86
87     — Переезд
88     WHEN (v_item.value)-->>'$type' = 'railway' THEN
89         — Поиск отправной станции
90         SELECT
91             s.id INTO v_station_from_id
92         FROM
93             station AS s
94         INNER JOIN
95             branch_station AS bs ON s.id = bs.station_id
96         WHERE
97             bs.branch_id = v_branch_id AND
98             s.title = (v_item.value)-->>'FromStationTitle';
99
100        — Поиск станции назначения
101        SELECT
102            s.id INTO v_station_to_id
103        FROM
104            station AS s
105        INNER JOIN
106            branch_station AS bs ON s.id = bs.station_id
107        WHERE
108            bs.branch_id = v_branch_id AND
109            s.title = (v_item.value)-->>'ToStationTitle';
110
111        — Поиск связываемого переезда
112        SELECT
113            r.id INTO v_railway_id
114        FROM
115            railway AS r
116        WHERE
117            (r.from_id = v_station_from_id AND r.to_id = v_station_to_id) OR
118            (r.from_id = v_station_to_id AND r.to_id = v_station_from_id);
119
120        IF v_railway_id IS NULL THEN
121            RAISE EXCEPTION 'На шаге % не найден переезд между станциями % и % на ветке %',
122                v_step_nomer, v_station_from_id, v_station_to_id, v_branch_id;
123        END IF;

```

Листинг 4.9 – Реализация функции добавления маршрута (часть 3)

```

124      — Вставка звена маршрута <Переезд>
125      INSERT INTO way_item_railway (way_item_id, railway_id)
126      VALUES (v_way_item_id, v_railway_id);
127
128      — Переход
129      WHEN (v_item.value)->>'<type>' = '<transition>' THEN
130      — Поиск первой связанной станции
131      SELECT
132          s.id INTO v_station_from_id
133      FROM
134          station AS s
135      WHERE
136          s.Title = (v_item.value)->>'<FromStationTitle>' AND
137          EXISTS(
138              SELECT
139                  1
140              FROM
141                  branch AS b
142              WHERE
143                  b.title = (v_item.value)->>'<FromBranchTitle>' AND
144                  EXISTS (
145                      SELECT
146                          1
147                      FROM
148                          branch_station AS bs
149                      WHERE
150                          bs.branch_id = b.id AND bs.station_id = s.id
151                  )
152          );
153
154      — Поиск второй связанной станции
155      SELECT
156          s.id INTO v_station_to_id
157      FROM
158          station AS s
159      WHERE
160          s.Title = (v_item.value)->>'<ToStationTitle>' AND
161          EXISTS(
162              SELECT
163                  1
164              FROM
165                  branch AS b
166              WHERE
167                  b.title = (v_item.value)->>'<ToBranchTitle>' AND
168                  EXISTS (
169                      SELECT
170                          1
171                      FROM
172                          branch_station AS bs
173                      WHERE
174                          bs.branch_id = b.id AND bs.station_id = s.id
175                  )
176          );
177
178      — Поиск связываемого перехода
179      SELECT
180          st.transition_id INTO v_transition_id
181      FROM
182          station_transition AS st
183      WHERE
184          st.station_id = v_station_from_id OR st.station_id =
185              v_station_to_id
186      GROUP BY
187          st.transition_id

```

Листинг 4.10 – Реализация функции добавления маршрута (часть 4)

```

187         HAVING
188             COUNT(*) = 2; — Только у подходящего перехода количество связей буд
                ет равно 2
189
190         — Вставка звена маршрута <Переход>
191         INSERT INTO way_item_transition (way_item_id, transition_id)
192         VALUES (v_way_item_id, v_transition_id);
193
194     END CASE;
195
196     v_step_nomer := v_step_nomer + 1;
197 END LOOP;
198
199 RETURN v_way_id;
200 END;
201 $$ LANGUAGE plpgsql;

```

Листинг 4.11 – Реализация функции получения схемы (часть 1)

```

1 CREATE OR REPLACE FUNCTION get_chart_json_by_id(chart_id INT)
2 RETURNS JSONB AS $$
3 DECLARE
4     result_json JSONB;
5 BEGIN
6     SELECT
7         jsonb_build_object(
8             'city', c.city,
9             'title', c.title,
10            'branches', COALESCE((
11                SELECT
12                    jsonb_agg(
13                        jsonb_build_object(
14                            'title', b.title,
15                            'color', UPPER((SELECT int_color_to_hex(b.color))),
16                            'accesstype', b.access,
17                            'stations', COALESCE((
18                                SELECT
19                                    jsonb_agg(
20                                        jsonb_build_object(
21                                            'title', s.title,
22                                            'occupancy', s.occupancy,
23                                            'accesstype', s.access,
24                                            'opentime', (SELECT TO_CHAR(s.open_time,
25                                                                    'HH24:MI')),
26                                            'closetime', (SELECT TO_CHAR(s.close_time,
27                                                                    'HH24:MI'))
28                                        )
29                                    )
30                                FROM
31                                    station AS s
32                                WHERE
33                                    EXISTS (
34                                        SELECT
35                                            1
36                                        FROM
37                                            branch_station AS bs
38                                        WHERE
39                                            bs.station_id = s.id AND bs.branch_id = b.id
40                                    )
41                                ), '[]'::JSONB),
42                            'railways', COALESCE((
43                                SELECT
44                                    jsonb_agg(
45                                        jsonb_build_object(

```

Листинг 4.12 – Реализация функции получения схемы (часть 2)

```

44         'from', (
45         SELECT
46             s.title
47         FROM
48             station AS s
49         WHERE
50             s.id = r.from_id
51         ),
52         'to', (
53         SELECT
54             s.title
55         FROM
56             station AS s
57         WHERE
58             s.id = r.to_id
59         ),
60         'duration', r.duration
61     )
62 )
63 FROM
64     railway AS r
65 WHERE
66     EXISTS (
67         SELECT
68             1
69         FROM
70             branch_station AS bs
71         JOIN
72             station AS s ON b.id = bs.branch_id AND s.id =
73                 bs.station_id AND s.id = r.from_id
74     ), '[]'::JSONB)
75 )
76 )
77 FROM
78     branch AS b
79 JOIN
80     chart_branch AS cb ON cb.chart_id = c.id AND cb.branch_id = b.id
81 ), '[]'::JSONB),
82 'transitions', COALESCE((
83 WITH
84     adj AS (
85         SELECT
86             bs.branch_id,
87             bs.station_id,
88             t.id AS transition_id,
89             t.occupancy,
90             t.access,
91             t.duration,
92             t.open_time,
93             t.close_time
94         FROM
95             chart_branch AS cb
96         JOIN
97             branch_station AS bs ON cb.chart_id = c.id AND cb.branch_id =
98                 bs.branch_id
99         JOIN
100             station_transition AS st ON st.station_id = bs.station_id
101         JOIN
102             transition AS t ON t.id = st.transition_id

```

Листинг 4.13 – Реализация функции получения схемы (часть 3)

```

103         r_adj AS (
104             SELECT
105                 adj.branch_id ,
106                 adj.station_id ,
107                 adj.transition_id ,
108                 rank() OVER (PARTITION BY transition_id ORDER BY branch_id ,
109                             station_id)
110             FROM
111                 adj
112         ),
113         d_adj AS (
114             SELECT DISTINCT ON (adj.transition_id)
115                 *
116             FROM
117                 adj
118             ORDER BY
119                 adj.transition_id
120         )
121     SELECT
122         jsonb_agg(
123             jsonb_build_object(
124                 'occupancy', d_adj.occupancy ,
125                 'accesstype', d_adj.access ,
126                 'duration', d_adj.duration ,
127                 'opentime', (SELECT TO_CHAR(d_adj.open_time, 'HH24:MI')),
128                 'closetime', (SELECT TO_CHAR(d_adj.close_time, 'HH24:MI')),
129                 'branchsrc', (
130                     SELECT
131                         b.title
132                     FROM
133                         r_adj
134                     JOIN
135                         branch AS b ON r_adj.transition_id =
136                             d_adj.transition_id AND b.id = r_adj.branch_id AND
137                             r_adj.rank = 1
138                 ),
139                 'stationsrc', (
140                     SELECT
141                         s.title
142                     FROM
143                         r_adj
144                     JOIN
145                         station AS s ON r_adj.transition_id =
146                             d_adj.transition_id AND s.id = r_adj.station_id
147                             AND r_adj.rank = 1
148                 ),
149                 'branchdst', (
150                     SELECT
151                         b.title
152                     FROM
153                         r_adj
154                     JOIN
155                         branch AS b ON r_adj.transition_id =
156                             d_adj.transition_id AND b.id = r_adj.branch_id AND
157                             r_adj.rank = 2
158                 )
159             )
160         )

```

Листинг 4.14 – Реализация функции получения схемы (часть 4)

```

152             'stationdst', (
153                 SELECT
154                     s.title
155                 FROM
156                     r_adj
157                 JOIN
158                     station AS s ON r_adj.transition_id =
159                                     d_adj.transition_id AND s.id = r_adj.station_id
160                                     AND r_adj.rank = 2
161             )
162         )
163     FROM
164         d_adj
165     ), '[]'::JSONB)
166 ) INTO result_json
167 FROM
168     chart AS c
169 WHERE
170     c.id = chart_id;
171 RETURN result_json;
172 END;
173 $$ LANGUAGE plpgsql

```

Листинг 4.15 – Реализация функции получения маршрута (часть 1)

```

1 CREATE OR REPLACE FUNCTION get_route_json_by_id(way_id INT)
2 RETURNS JSONB AS $$
3 DECLARE
4     result_json JSONB;
5 BEGIN
6     SELECT
7         jsonb_build_object(
8             'City', c.city,
9             'ChartTitle', c.title,
10            'Title', w.title,
11            'Duration', w.duration,
12            'RouteItems', COALESCE((
13                WITH
14                    wi AS (
15                        SELECT
16                            way_item.id,
17                            way_item.way_id,
18                            way_item.nexus,
19                            way_item.step_nomer
20                        FROM
21                            way_item
22                        WHERE
23                            way_item.way_id = w.id
24                    ),
25                    ris AS (
26                        SELECT
27                            wis.way_item_id,
28                            wis.station_id,
29                            bs.branch_id,
30                            s.title AS station_title,
31                            b.title AS branch_title,
32                            s.occupancy AS station_occupancy,
33                            s.access AS station_access,
34                            s.open_time AS station_open_time,
35                            s.close_time AS station_close_time,
36                            wi.step_nomer

```


Листинг 4.16 – Реализация функции получения маршрута (часть 2)

```

37         FROM
38             wi
39         INNER JOIN
40             way_item_station AS wis ON wi.id = wis.way_item_id
41         INNER JOIN
42             station AS s ON s.id = wis.station_id
43         INNER JOIN
44             branch_station AS bs ON bs.station_id = s.id
45         INNER JOIN
46             branch AS b ON b.id = bs.branch_id
47     ),
48     rir AS (
49         SELECT
50             way_item_id,
51             branch_title,
52             from_station_title,
53             to_station_title,
54             railway_duration,
55             step_nomer
56         FROM (
57             SELECT
58                 wir.way_item_id,
59                 wi.nexus,
60                 LAG(ris.branch_title) OVER (ORDER BY wi.step_nomer) AS
61                     branch_title,
62                 LAG(ris.station_title) OVER (ORDER BY wi.step_nomer)
63                     AS from_station_title,
64                 LEAD(ris.station_title) OVER (ORDER BY wi.step_nomer)
65                     AS to_station_title,
66                 r.duration AS railway_duration,
67                 wi.step_nomer
68             FROM
69                 wi
70             LEFT JOIN
71                 ris ON wi.id = ris.way_item_id
72             LEFT JOIN
73                 way_item_railway AS wir ON wi.id = wir.way_item_id
74             LEFT JOIN
75                 railway AS r ON wir.railway_id = r.id
76         )
77         WHERE
78             nexus = 'RAILWAY'::nexus_type
79     ),
80     rit AS (
81         SELECT
82             way_item_id,
83             from_station_title,
84             from_branch_title,
85             to_station_title,
86             to_branch_title,
87             transition_occupancy,
88             transition_access,
89             transition_duration,
90             transition_open_time,
91             transition_close_time,
92             step_nomer

```

Листинг 4.17 – Реализация функции получения маршрута (часть 3)

```

90         FROM (
91             SELECT
92                 wit.way_item_id,
93                 wi.nexus,
94                 LAG(ris.station_title) OVER (ORDER BY wi.step_nomer)
95                     AS from_station_title,
96                 LAG(ris.branch_title) OVER (ORDER BY wi.step_nomer) AS
97                     from_branch_title,
98                 LEAD(ris.station_title) OVER (ORDER BY wi.step_nomer)
99                     AS to_station_title,
100                 LEAD(ris.branch_title) OVER (ORDER BY wi.step_nomer)
101                     AS to_branch_title,
102                 t.occupancy AS transition_occupancy,
103                 t.access AS transition_access,
104                 t.duration AS transition_duration,
105                 t.open_time AS transition_open_time,
106                 t.close_time AS transition_close_time,
107                 wi.step_nomer
108             FROM
109                 wi
110             LEFT JOIN
111                 ris ON wi.id = ris.way_item_id
112             LEFT JOIN
113                 way_item_transition AS wit ON wi.id = wit.way_item_id
114             LEFT JOIN
115                 transition AS t ON wit.transition_id = t.id
116             WHERE
117                 wi.nexus != 'RAILWAY'::nexus_type
118         )
119     WHERE
120         nexus = 'TRANSITION'::nexus_type
121 ),
122 res AS (
123     SELECT
124         wi.step_nomer,
125         wi.nexus,
126         CASE wi.nexus
127             WHEN 'STATION'::nexus_type THEN ris.station_title
128             ELSE NULL
129         END AS station_title,
130         CASE wi.nexus
131             WHEN 'STATION'::nexus_type THEN ris.branch_title
132             WHEN 'RAILWAY'::nexus_type THEN ris.branch_title
133             ELSE NULL
134         END AS branch_title,
135         CASE wi.nexus
136             WHEN 'STATION'::nexus_type THEN ris.station_occupancy
137             ELSE NULL
138         END AS station_occupancy,
139         CASE wi.nexus
140             WHEN 'STATION'::nexus_type THEN ris.station_access
141             ELSE NULL
142         END AS station_access,
143         CASE wi.nexus
144             WHEN 'STATION'::nexus_type THEN ris.station_open_time
145             ELSE NULL
146         END AS station_open_time,
147         CASE wi.nexus
148             WHEN 'STATION'::nexus_type THEN ris.station_close_time
149             ELSE NULL
150         END AS station_close_time,

```

Листинг 4.18 – Реализация функции получения маршрута (часть 4)

```

147         CASE wi.nexus
148             WHEN 'RAILWAY'::nexus_type THEN rir.from_station_title
149             WHEN 'TRANSITION'::nexus_type THEN
150                 rit.from_station_title
151             ELSE NULL
152         END AS from_station_title,
153         CASE wi.nexus
154             WHEN 'RAILWAY'::nexus_type THEN rir.to_station_title
155             WHEN 'TRANSITION'::nexus_type THEN rit.to_station_title
156             ELSE NULL
157         END AS to_station_title,
158         CASE wi.nexus
159             WHEN 'RAILWAY'::nexus_type THEN rir.railway_duration
160             ELSE NULL
161         END AS railway_duration,
162         CASE wi.nexus
163             WHEN 'TRANSITION'::nexus_type THEN
164                 rit.from_branch_title
165             ELSE NULL
166         END AS from_branch_title,
167         CASE wi.nexus
168             WHEN 'TRANSITION'::nexus_type THEN rit.to_branch_title
169             ELSE NULL
170         END AS to_branch_title,
171         CASE wi.nexus
172             WHEN 'TRANSITION'::nexus_type THEN
173                 rit.transition_occupancy
174             ELSE NULL
175         END AS transition_occupancy,
176         CASE wi.nexus
177             WHEN 'TRANSITION'::nexus_type THEN
178                 rit.transition_access
179             ELSE NULL
180         END AS transition_access,
181         CASE wi.nexus
182             WHEN 'TRANSITION'::nexus_type THEN
183                 rit.transition_duration
184             ELSE NULL
185         END AS transition_duration,
186         CASE wi.nexus
187             WHEN 'TRANSITION'::nexus_type THEN
188                 rit.transition_open_time
189             ELSE NULL
190         END AS transition_open_time,
191         CASE wi.nexus
192             WHEN 'TRANSITION'::nexus_type THEN
193                 rit.transition_close_time
194             ELSE NULL
195         END AS transition_close_time
196     FROM
197         wi
198     LEFT JOIN
199         ris ON wi.id = ris.way_item_id
200     LEFT JOIN
201         rir ON wi.id = rir.way_item_id
202     LEFT JOIN
203         rit ON wi.id = rit.way_item_id
204     ORDER BY
205         wi.step_nomer
206 )
207 SELECT
208     jsonb_agg(

```

Листинг 4.19 – Реализация функции получения маршрута (часть 5)

```

202         CASE res.nexus
203             WHEN 'STATION'::nexus_type THEN
204                 jsonb_build_object(
205                     '$type', 'station',
206                     'Title', res.station_title,
207                     'BranchTitle', res.branch_title,
208                     'Occupancy', res.station_occupancy,
209                     'Access', res.station_access,
210                     'OpenTime', res.station_open_time,
211                     'CloseTime', res.station_close_time
212                 )
213             WHEN 'RAILWAY'::nexus_type THEN
214                 jsonb_build_object(
215                     '$type', 'railway',
216                     'BranchTitle', res.branch_title,
217                     'FromStationTitle', res.from_station_title,
218                     'ToStationTitle', res.to_station_title,
219                     'Duration', res.railway_duration
220                 )
221             WHEN 'TRANSITION'::nexus_type THEN
222                 jsonb_build_object(
223                     '$type', 'transition',
224                     'FromBranchTitle', res.from_branch_title,
225                     'FromStationTitle', res.from_station_title,
226                     'ToBranchTitle', res.to_branch_title,
227                     'ToStationTitle', res.to_station_title,
228                     'Occupancy', res.transition_occupancy,
229                     'Access', res.transition_access,
230                     'Duration', res.transition_duration,
231                     'OpenTime', res.transition_open_time,
232                     'CloseTime', res.transition_close_time
233                 )
234             ELSE
235                 jsonb_build_object('$type', 'unknown')
236         END
237     )
238     FROM
239         res
240     ), '[]'::JSONB)
241 ) INTO result_json
242 FROM
243     way AS w
244 INNER JOIN
245     chart AS c ON w.chart_id = c.id
246 WHERE
247     w.id = way_id;
248
249
250     RETURN result_json;
251 END;
252 $$ LANGUAGE plpgsql

```

Листинг 4.20 – Реализация функций конвертации цвета

```

1 CREATE OR REPLACE FUNCTION hex_color_to_int(hex_color TEXT)
2 RETURNS INT AS $$
3 DECLARE
4     cleaned_hex TEXT;
5     full_hex TEXT;
6     color_int INT;
7 BEGIN
8     cleaned_hex := UPPER(TRIM(BOTH '#' FROM hex_color));
9
10    IF LENGTH(cleaned_hex) = 3 THEN
11        full_hex := CONCAT(
12            SUBSTR(cleaned_hex, 1, 1), SUBSTR(cleaned_hex, 1, 1),
13            SUBSTR(cleaned_hex, 2, 1), SUBSTR(cleaned_hex, 2, 1),
14            SUBSTR(cleaned_hex, 3, 1), SUBSTR(cleaned_hex, 3, 1)
15        );
16    ELSIF LENGTH(cleaned_hex) = 6 THEN
17        full_hex := cleaned_hex;
18    ELSE
19        RAISE EXCEPTION 'Неверный_формат_hex-цвета:_%', hex_color;
20    END IF;
21
22    EXECUTE 'SELECT_' || quote_literal('x' || full_hex) || '::_bit(24)::int'
23    INTO color_int;
24
25    RETURN color_int;
26 END;
27 $$ LANGUAGE plpgsql IMMUTABLE;
28
29
30 CREATE OR REPLACE FUNCTION int_color_to_hex(int_color INT)
31 RETURNS TEXT AS $$
32 DECLARE
33     hex_color TEXT;
34 BEGIN
35     IF int_color IS NULL OR int_color < 0 OR int_color > 16777215 THEN
36         RAISE EXCEPTION 'Цвет_должен_быть_в_диапазоне_от_0_до_16777215,_получено:_%',
37             int_color;
38     END IF;
39
40     hex_color := LPAD(TO_HEX(int_color), 6, '0');
41
42     RETURN '#' || LOWER(hex_color);
43 END;
44 $$ LANGUAGE plpgsql IMMUTABLE;

```

Листинг 4.21 – Реализация навигационных функций

```

1 CREATE OR REPLACE FUNCTION get_station_id_by_branch_id_station_title(
2     p_branch_id INT,
3     station_title TEXT
4 ) RETURNS INT AS $$
5 DECLARE
6     v_station_id INT;
7 BEGIN
8     SELECT
9         s.id INTO v_station_id
10    FROM
11        branch_station AS bs
12    INNER JOIN
13        station AS s ON bs.station_id = s.id
14    WHERE
15        bs.branch_id = p_branch_id AND
16        s.title = station_title;
17
18    RETURN v_station_id;
19 END;
20 $$ LANGUAGE plpgsql STABLE;
21
22
23 CREATE OR REPLACE FUNCTION get_station_id_by_branch_title_station_title(
24     branch_title TEXT,
25     station_title TEXT
26 ) RETURNS INT AS $$
27 DECLARE
28     v_station_id INT;
29 BEGIN
30     SELECT
31         s.id INTO v_station_id
32    FROM
33        branch_station AS bs
34    INNER JOIN
35        branch AS b ON bs.branch_id = b.id
36    INNER JOIN
37        station AS s ON bs.station_id = s.id
38    WHERE
39        b.title = branch_title AND
40        s.title = station_title;
41
42    RETURN v_station_id;
43 END;
44 $$ LANGUAGE plpgsql STABLE;

```

Листинг 4.22 – Реализация проверяющих триггерных функций (часть 1)

```

1  — Проверка корректного добавления связи Схема—Ветка
2  CREATE OR REPLACE FUNCTION ck_chart_branch_unique_ref() RETURNS TRIGGER
3  AS $$
4  BEGIN
5      IF EXISTS (
6          SELECT
7              1
8          FROM
9              chart_branch
10         WHERE
11             chart_id <> NEW.chart_id AND branch_id = NEW.branch_id
12     ) THEN
13         RAISE EXCEPTION
14             'Попытка_связать_ветку_(%)_уже_связанную_со_схемой_(%)_со_схемой_(%)_ ',
15             branch_id, chart_id, NEW.chart_id;
16     END IF;
17     RETURN NEW;
18 END;
19 $$ LANGUAGE plpgsql;
20
21 — Проверка корректного добавления связи Ветка—Станция
22 CREATE OR REPLACE FUNCTION ck_branch_station_unique_ref() RETURNS TRIGGER
23 AS $$
24 BEGIN
25     IF EXISTS (
26         SELECT
27             1
28         FROM
29             branch_station
30         WHERE
31             branch_id <> NEW.branch_id AND station_id = NEW.station_id
32     ) THEN
33         RAISE EXCEPTION
34             'Попытка_связать_станцию_(%)_уже_связанную_с_веткой_(%)_с_веткой_(%)_ ',
35             station_id, branch_id, NEW.branch_id;
36     END IF;
37     RETURN NEW;
38 END;
39 $$ LANGUAGE plpgsql;
40
41 — Проверка дублирования переездов между станциями
42 CREATE OR REPLACE FUNCTION ck_railway_unique_station_ref() RETURNS TRIGGER
43 AS $$
44 BEGIN
45     IF EXISTS (
46         SELECT
47             1
48         FROM
49             railway
50         WHERE
51             from_id = NEW.to_id AND to_id = NEW.from_id
52     ) THEN
53         RAISE EXCEPTION
54             'Дублирование_переезда_между_станциями_Станции_(%,%)_уже_соединены_переез
55             дом:_(%,%)_ ',
56             NEW.from_id, NEW.to_id, from_id, to_id;
57     END IF;
58     RETURN NEW;
59 END;
60 $$ LANGUAGE plpgsql;

```

Листинг 4.23 – Реализация проверяющих триггерных функций (часть 2)

```

61 — Проверка связывания станций переездом в рамках одной ветки
62 CREATE OR REPLACE FUNCTION ck_railway_same_branch_of_stations_ref() RETURNS TRIGGER
63 AS $$
64 DECLARE
65     branch1_id INT;
66     branch2_id INT;
67 BEGIN
68     SELECT
69         branch_id INTO branch1_id
70     FROM
71         branch_station AS bs
72     WHERE
73         NEW.from_id = bs.station_id
74     LIMIT 1;
75
76     SELECT
77         branch_id INTO branch2_id
78     FROM
79         branch_station AS bs
80     WHERE
81         NEW.to_id = bs.station_id
82     LIMIT 1;
83
84     IF branch1_id IS NULL THEN
85         RAISE EXCEPTION
86             'Станция_%непривязана_ни_к_одной_ветке',
87             NEW.from_id;
88     ELSIF branch2_id IS NULL THEN
89         RAISE EXCEPTION
90             'Станция_%непривязана_ни_к_одной_ветке',
91             NEW.to_id;
92     ELSIF branch1_id <> branch2_id THEN
93         RAISE EXCEPTION
94             'Попытка_соединения_переездом_станций_(%,%)_принадлежащие_разным_веткам:_(%,%)',
95             NEW.from_id, NEW.to_id, branch1_id, branch2_id;
96     END IF;
97
98     RETURN NEW;
99 END;
100 $$ LANGUAGE plpgsql;
101
102
103 — Проверка связывания станций переходом в рамках разных веток одной и той же схемы
104 CREATE OR REPLACE FUNCTION ck_station_transition_different_branch_of_stations_ref()
105 RETURNS TRIGGER
106 AS $$
107 DECLARE
108     transition_links INT;
109     station1_id INT;
110     station2_id INT;
111     branch1_id INT;
112     branch2_id INT;
113     chart1_id INT;
114     chart2_id INT;
115 BEGIN
116     — Подсчет количества станций ведущих на один и тот же переход
117     SELECT
118         count(st.transition_id) INTO transition_links
119     FROM
120         station_transition AS st
121     WHERE
122         st.transition_id = NEW.transition_id;

```


Листинг 4.24 – Реализация проверяющих триггерных функций (часть 3)

```

123 — Гарантия, что только две станции ведут на переход
124 IF transition_links < 2 THEN
125     RETURN NEW; — переход еще не полностью установлен
126 ELSIF transition_links > 2 THEN
127     RAISE EXCEPTION
128         'Попытка связать переход(%) с больше чем 2-мя станциями',
129         NEW.transition_id;
130 END IF;
131
132 — Первая станция уже имеется в новой записи NEW
133 SELECT
134     NEW.station_id INTO station1_id;
135
136 — Вторая станция избирается с учетом того, что всего станций 2 и проверкой
137 — на несовпадение с NEW.station_id
138 SELECT
139     st.station_id INTO station2_id
140 FROM
141     station_transition AS st
142 WHERE
143     st.transition_id = NEW.transition_id AND st.station_id <> NEW.station_id
144 LIMIT 1;
145
146 — Вторая станция должна быть найдена
147 IF station2_id IS NULL THEN
148     RAISE EXCEPTION
149         'Не удалось определить вторую станцию для перехода%',
150         NEW.transition_id;
151 END IF;
152
153 — Поиск ветки и схемы для первой станции
154 SELECT
155     bs.branch_id, cb.chart_id INTO branch1_id, chart1_id
156 FROM
157     branch_station bs
158 LEFT JOIN
159     chart_branch cb ON cb.branch_id = bs.branch_id
160 WHERE
161     bs.station_id = station1_id
162 LIMIT 1;
163
164 — Поиск ветки и схемы для второй станции
165 SELECT
166     bs.branch_id, cb.chart_id INTO branch2_id, chart2_id
167 FROM
168     branch_station bs
169 LEFT JOIN
170     chart_branch cb ON cb.branch_id = bs.branch_id
171 WHERE
172     bs.station_id = station2_id
173 LIMIT 1;
174
175 IF branch1_id IS NULL THEN
176     RAISE EXCEPTION
177         'Станция(%) не привязана к ветке',
178         station1_id;
179 ELSIF branch2_id IS NULL THEN
180     RAISE EXCEPTION
181         'Станция(%) не привязана к ветке',
182         station2_id;
183 ELSIF chart1_id IS NULL THEN
184     RAISE EXCEPTION
185         'Ветка(%) не привязана к схеме',
186         branch1_id;

```

Листинг 4.25 – Реализация проверяющих триггерных функций (часть 4)

```

187     ELSIF chart2_id IS NULL THEN
188         RAISE EXCEPTION
189             'Ветка_(%)_не_привязана_к_схеме',
190             branch2_id;
191     ELSIF branch1_id = branch2_id THEN
192         RAISE EXCEPTION
193             'Попытка_соединения_переходом_станций_(%,_)_,_принадлежащие_одной_и_той_же_
              ветке:_(%)_.',
194             statoin1_id, station2_id, branch1_id;
195     END IF;
196
197     RETURN NEW;
198 END;
199 $$ LANGUAGE plpgsql;

```

Листинг 4.26 – Реализация удаляющих триггерных функций

```

1  — Удаление всех branch, которые были связаны с удаляемым chart
2  CREATE OR REPLACE FUNCTION delete_orphaned_branches_before_chart_delete()
3  RETURNS TRIGGER AS $$
4  BEGIN
5      DELETE FROM branch
6      WHERE id IN (
7          SELECT cb.branch_id
8          FROM chart_branch AS cb
9          WHERE cb.chart_id = OLD.id
10     );
11
12     RETURN OLD;
13 END;
14 $$ LANGUAGE plpgsql;
15
16
17 — Удаление всех Station, которые были связаны с удаляемым Branch
18 CREATE OR REPLACE FUNCTION delete_orphaned_stations_before_branch_delete()
19 RETURNS TRIGGER AS $$
20 BEGIN
21     DELETE FROM station
22     WHERE id IN (
23         SELECT bs.station_id
24         FROM branch_station AS bs
25         WHERE bs.branch_id = OLD.id
26     );
27
28     RETURN OLD;
29 END;
30 $$ LANGUAGE plpgsql;
31
32
33 — Удаление всех Transition, которые были связаны с удаляемым Station
34 CREATE OR REPLACE FUNCTION delete_orphaned_transitions_before_station_delete()
35 RETURNS TRIGGER AS $$
36 BEGIN
37     DELETE FROM transition
38     WHERE id IN (
39         SELECT st.transition_id
40         FROM station_transition AS st
41         WHERE st.station_id = OLD.id
42     );
43
44     RETURN OLD;
45 END;
46 $$ LANGUAGE plpgsql;

```